Generative Adversarial Networks

Experimenting with GAN Architectures on the Pokemon dataset

Rania Hamid

I. Introduction

Objective of the Study

The objective of this study is to implement, train, and systematically tune a Generative Adversarial Network (GAN) capable of generating synthetic Pokémon sprite images. Specifically, this analysis investigates how loss function choice, architectural capacity, and regularization strength interact to determine GAN training stability and image quality. The central question guiding the study is: to what extent can hyperparameter tuning of a DCGAN-derived architecture produce stable, high-quality image synthesis on a small, visually complex dataset?

Background and Problem Domain

Generative Adversarial Networks have found meaningful application across industries where real data is scarce, sensitive, or insufficient for downstream modeling. In healthcare, GANs synthesize medical images to augment training sets for diagnostic models where labeled data is expensive and privacy-constrained (Frid-Adar et al., 2018). In financial services, they generate synthetic transaction records for fraud detection without exposing customer data (Fiore et al., 2019). In manufacturing, they produce synthetic defect images for quality inspection systems where real defect examples are rare by design (Niu et al., 2020). Across these domains, the core value proposition is the same: when real data is unavailable or restricted, a well-trained generative model can approximate the underlying distribution closely enough to serve as a proxy.

A particularly relevant application is in the energy sector. Grid operators face a growing challenge in modeling large, novel electric loads from data centers, crypto mining operations, and utility-scale EV charging infrastructure - representative load profile data either does not yet exist or is proprietary (Large Load Technical Conference, 2026). Conditional GANs, which condition generation on a class label or auxiliary input (Mirza & Osindero, 2014), trained on historical data from analogous facilities could generate the synthetic profiles that grid planners need to assess interconnection impacts and plan capacity — a gap explicitly identified in recent industry assessments (NERC, 2025).

This project uses Pokémon sprite synthesis as a tractable proxy for this broader challenge. The dataset is small enough to train on consumer-grade hardware, visually complex enough to stress-test GAN stability, and structured enough to support rigorous evaluation. The goal is a controlled study on the sensitivity of GANs to hyperparameter choices, a skill that transfers directly to producing synthetic data applications regardless of domain.

Method Rationale

GANs are well-suited for synthetic image generation because they learn to capture the full distribution of the training data rather than simply minimizing pixel-level reconstruction error (Goodfellow et al., 2014). Training proceeds through an adversarial dynamic between a generator, which synthesizes images from random noise, and a discriminator, which attempts to tell real images from generated ones. Each network's improvements pressure the other to improve, creating a feedback loop that can, in principle, produce generated images indistinguishable from real ones.

The Deep Convolutional GAN (DCGAN) extends this framework to image data by replacing fully connected layers with convolutional operations, allowing the network to exploit the spatial structure of images rather than treating pixels as independent inputs (Radford et al., 2015). This makes DCGAN a natural fit for the 64×64 RGB sprite images in this dataset. It serves as the architectural starting point for all experiments, with loss function and capacity modifications applied progressively as training limitations emerged.

II. Analysis

Figure 1: Random Sample of Sprites from Pokemon Dataset

Dataset Description

- **Dataset name**: Pokemon Sprite Dataset
- **Source**: Dive into Deep Learning (d2l) data repository, originally sourced from PokemonDB
- **Input features**: PNG “sprite” images, palette mode (P), variable resolution from 40x29 to 142x99 pixels, normalized to 64x64 RGB arrays in the range [-1, 1]
- **Labels**: Integer class indices corresponding to Pokedex species numbers (0-720); these labels were not used during GAN training.

The dataset consists of 40,597 images distributed across 721 classes, where each class corresponds to one Pokemon species. Multiple images per class represent different sprites of the

same species across game generations, including front and back views, shiny color variants, and form differences. The mean number of images per class is 56.3 with a median of 59.0.

Exploratory Data Analysis (EDA)

Initial exploration of the dataset was conducted to understand its structure and contents.

Sprites varied in shape, size, and color intensities. Resolution sizes range from 40x29 to 142x99 pixels. The low resolution produces heavily pixelated images where fine details are lost. Later generation pokemon sprites had notably better resolution than earlier generation pokemon.

Class imbalance (Appendix A – Figure 1)

Class counts range from a minimum of 1 to a maximum of 239, representing a 239:1 imbalance ratio. Classes with very few images risk the generator memorizing specific examples rather than learning a generalizable distribution. A threshold of 30 images per class was applied prior to training, retaining 669 of 721 classes and 39,285 of 40,597 images. This threshold removes only 52 classes (7% of species) while preserving 97% of total images, and classes below the threshold are disproportionately likely to contain stylistically inconsistent sprites from a single game generation. Plots of the class distribution are included in Appendix A.

Pixel Statistics and Background Dominance (Appendix A – Figure 2)

All images are stored in palette mode (P), meaning each pixel is an index into a color table rather than a direct RGB value — requiring explicit conversion before the data can be fed into a neural network. Pixel analysis on a sample of 1,000 images revealed severe background dominance: nearly half of all pixels (48.3%) were near-white (RGB > 240/255), with channel means of 0.813, 0.797, and 0.779 on a 0–1 scale. The pixel intensity histogram (Appendix A) shows a sharp spike at pure white that far outweighs the subject pixel distribution. Left unaddressed, the generator would be incentivized to learn the background rather than the Pokemon. This was confirmed when a baseline DCGAN run on the unprocessed dataset produced no recognizable structure after full training, establishing that background removal was a prerequisite for meaningful learning.

Transparency Encoding (Appendix A – Figure 4)

62.8% of images encode transparency, with pixels being either fully transparent or fully opaque. The remaining 37.2% carry no transparency metadata at all — for these, converting to RGBA produces an alpha channel of zero everywhere, making it useless for background detection. Visual inspection confirmed that these images consistently have white backgrounds, making color thresholding a reliable fallback.

Image Size Distribution (Appendix A – Figure 5)

Original image sizes range from 40×29 to 142×99 pixels, with a median width of 72px and height of 59px. Two natural size clusters reflect the generational progression of the games — older sprites cluster around 40–60px wide and newer ones around 75–110px wide, with a small outlier group above 130px. Since images are consistently wider than tall, resizing directly to a square target would distort proportions, compounding the background dominance problem already identified.

Data Preprocessing

Each identified challenge from the EDA drove a specific preprocessing decision.

Background removal was applied to every image using one of two detection strategies depending on image type: alpha channel masking for images with transparency encoded, and white pixel color thresholding (RGB > 240) for those without. A 5% margin was added around the detected subject boundary before cropping to avoid clipping thin or elongated sprites too tightly — margins of 5%, 10%, and 15% produced visually identical results at 64×64, so the smallest was used.

Crop-and-pad was chosen over direct resizing after empirical comparison showed that direct resizing visibly distorted non-square sprites. The crop-and-pad approach crops tightly to the subject, adds symmetric white padding to make the canvas square, then resizes to 64×64. All images were composited over a white background before processing to ensure consistent handling of both transparent and opaque inputs.

The 64×64 target resolution was kept to maintain compatibility with the DCGAN architecture, which upsamples through four stages from a 4×4 spatial seed and requires power-of-2 dimensions. LANCZOS resampling was used throughout. Pixel values were normalized to [−1, 1] to match the output range of the generator's tanh activation.

The TensorFlow data pipeline was configured with shuffling, parallel preprocessing, a batch size of 256, caching, and prefetching, producing 153 batches per epoch.

Algorithm Intuition and Baseline Model Fitting

GAN Framework

A GAN frames image synthesis as a two-player minimax game between a generator G and a discriminator D (Goodfellow et al., 2014). The generator maps a random noise vector, sampled from a prior distribution, to a synthetic image. The discriminator maps an image, either real or generated, to a scalar judgment. Under the original binary cross-entropy framework, D is trained to output 1 for real images and 0 for generated images, while G is trained to fool D into outputting 1 for its outputs. Training alternates between updating D to minimize its classification error and updating G to maximize D's error on generated samples.

The adversarial dynamic creates a feedback loop in which each network's improvements pressure the other to improve further. In principle, if both networks have sufficient capacity and training is stable, the generator converges to a distribution indistinguishable from the real data distribution and the discriminator converges to outputting 0.5 everywhere. In practice, this equilibrium is notoriously difficult to achieve — particularly under BCE loss, where a confident discriminator produces vanishing gradients that provide no useful learning signal to the generator.

DCGAN Architecture

The Deep Convolutional GAN replaces the fully connected layers of the original GAN with convolutional operations (Radford et al., 2015). The generator uses transposed convolutions to progressively upsample a 1×1×100 latent vector through four G_blocks — each consisting of a Conv2DTranspose layer, batch normalization, and ReLU activation — to a 64×64×3 output, with a final tanh activation to constrain pixel values to [−1, 1]. Starting from n_G = 64 base filters, the channel progression descends as 512 → 256 → 128 → 64 → 3, concentrating representational capacity in early upsampling stages where spatial structure is first established.

The discriminator uses strided convolutions to progressively downsample a 64×64×3 input through four D_blocks — each consisting of a Conv2D layer, batch normalization, and LeakyReLU activation (alpha = 0.2) — to a 1×1×1 scalar output. LeakyReLU is used rather than ReLU to allow small gradients for negative inputs, which has been empirically shown to improve discriminator training stability in image synthesis tasks (Radford et al., 2015). All weights are initialized from a normal distribution with mean 0 and standard deviation 0.02, following the original DCGAN specification. Both networks are compiled with Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$), with the reduced β_1 preventing momentum from accumulating during the adversarial oscillations typical of early GAN training.

Wasserstein GAN with Gradient Penalty (WGAN-GP)

The Wasserstein GAN replaces the binary classification objective with an approximation of the Wasserstein-1 distance (also called the Earth Mover's distance) between the real and generated distributions (Arjovsky et al., 2017). The key insight is that BCE produces vanishing gradients when the discriminator is too confident whereas the Wasserstein distance provides meaningful gradients even when the two distributions have little overlap. The discriminator is renamed the "critic" because it no longer outputs a probability; instead it outputs a raw real-valued score, with higher scores assigned to more "real-like" images.

The original WGAN enforces its mathematical requirements through weight clipping: after every critic update, all critic weights are forcibly clamped to a narrow range (between −0.01 and 0.01). This keeps the critic well-behaved in a theoretical sense, but in practice it tends to squeeze too much of the critic's learning capacity into a small number of dominant patterns, which pushes the

generator to exploit those patterns and produce repetitive outputs — a form of mode collapse. WGAN-GP (Gulrajani et al., 2017) fixes this by replacing weight clipping with a gradient penalty: instead of constraining the weights directly, it adds a term to the critic's training objective that penalizes the critic whenever its output changes too sharply or too slowly between real and generated images. This achieves the same theoretical requirement in a softer, more flexible way that does not bottle up the critic's capacity. The gradient penalty strength is controlled by a coefficient λ, which became a key tuning parameter in this study. Batch normalization is also removed from the critic because it creates dependencies between images in the same training batch that interfere with how the gradient penalty is computed. The critic is updated five times for every one generator update to ensure it stays well-trained and provides a reliable learning signal to the generator.

Key Hyperparameters:

The latent dimension was fixed at 100 throughout all experiments, following the standard DCGAN specification. Higher-dimensional latent spaces can in principle support more diverse generation and would have been a natural axis of exploration, but compute constraints precluded this. The gradient penalty coefficient λ and the base filter count n were the primary tuned parameters in the WGAN-GP phase. The learning rate was held at 0.0001 across all WGAN-GP experiments after the learning rate adjustment experiment in the BCE phase demonstrated that differential learning rates did not resolve discriminator dominance.

Experimental Design and Model Fitting

Each experiment was designed to change one thing at a time, keeping all other settings fixed so that any change in performance could be attributed to that specific decision. This discipline is especially important in GAN training, where the generator and discriminator are constantly reacting to each other. When multiple variables shift at once, it becomes very difficult to know what actually caused the outcome.

Compute was a persistent constraint. All experiments ran on a single GPU in Google Colab. WGAN-GP runs were considerably slower than the DCGAN experiments, with roughly 390 examples per second versus over 1,000. This is due to the five critic updates per generator step and the gradient penalty overhead. A single 40-epoch run at the larger network size took about an hour, which ruled out grid searches, multi-seed averaging, and exploration of additional hyperparameter axes. Practicing deliberate, hypothesis-driven tuning under these constraints is itself a valuable exercise, building the kind of careful intuition for how GAN parameters interact that scales to larger, real-world applications.

DCGAN with Binary Cross-Entropy Loss

The baseline DCGAN was trained for 40 epochs with a learning rate of 0.0005 for both networks. The discriminator loss stabilized around 0.35 while the generator loss remained high and oscillating. Generated images showed some Pokemon-like structure with varied colors, visible limbs, and distinct body shapes, and no mode collapse. This result was only possible after preprocessing; a diagnostic run on the unprocessed dataset produced no recognizable structure, confirming that background removal and normalization were prerequisites.

The loss dynamics suggested the discriminator was pulling ahead, so a follow-up experiment halved its learning rate (lr_D = 0.00025, lr_G = 0.0005).

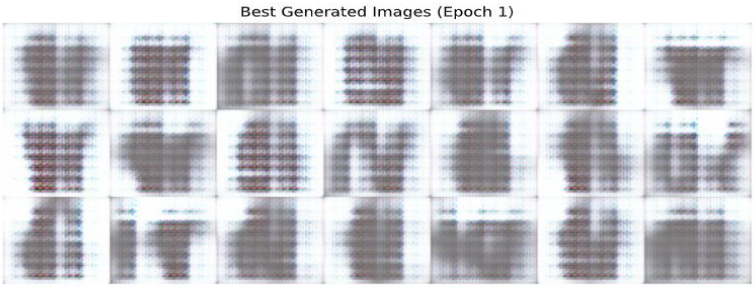

Figure 2: Image of 'best epoch' from DCGAN lr experiment

The discriminator collapsed even faster, dominating from the very first batch and ending training at loss_D = 0.077 and loss_G = 4.352. Despite this, generated image quality was comparable to the baseline — an early sign that BCE loss curves are a poor indicator of visual output quality.

Label smoothing (smooth = 0.9) was tried next, replacing hard real labels of 1.0 with soft labels of 0.9 to reduce discriminator overconfidence. The generator collapsed into a single repeating pattern by epoch 3, with loss_G falling to 0.002. Taken together, these three experiments reflect a well-documented failure mode of BCE-based GAN training (Arjovsky et al., 2017): once either network gains a decisive advantage, the loss signal degrades and no straightforward intervention reliably restores equilibrium.

WGAN with Weight Clipping

The transition to Wasserstein loss was motivated by the BCE phase's consistent failure to maintain a stable adversarial balance. The original WGAN (Arjovsky et al., 2017) was implemented with weight clipping (clip = 0.01), RMSprop optimizer, and n_critic = 5. Training appeared stable — the best checkpoint was recorded at epoch 26 with a near-zero loss gap — but visual inspection of generated images revealed a hallmark of WGAN weight clipping: mode collapse to a single repeating output pattern despite numerically healthy loss curves. This confirmed the known weakness of weight clipping: constraining all critic weights to a narrow range causes the critic's gradients to become dominated by a few dimensions, pushing the generator toward a single output. The WGAN-GP was implemented to address this.

WGAN-GP Baseline and Capacity Tuning

The WGAN-GP baseline was trained with n=64, λ=10, lr=0.0001, n_critic=5, and latent_dim=100 for 40 epochs. FID was computed every 10 epochs using a fixed cache of 500 real image InceptionV3 features extracted once before training and reused across all experiments. The baseline achieved its best FID of 739.79 at epoch 40, indicating continued improvement throughout training rather than convergence. Generated images exhibited diverse color palettes and no mode collapse, but displayed a watercolor-like appearance with soft edges suggesting insufficient representational capacity.

Tuning Experiment 1 increased the base filter count from n=64 to n=128 for both generator and critic, holding all other hyperparameters constant. FID worsened to 998.10, with the best checkpoint at epoch 30. The hypothesis was that λ=10, calibrated for a smaller network, over-constrained the larger n=128 critic and limited its ability to develop sharp decision boundaries.

Tuning Experiment 2 reduced λ from 10 to 7 while retaining n=128. A 10-epoch diagnostic run was conducted first to verify training stability before committing to a full 40-epoch run. The full run improved FID to 850.91, but the best checkpoint occurred at epoch 20, suggesting the generator began to drift in later epochs under the looser constraint. FID of 850.91 was still above the n=64 baseline of 739.79.

Tuning Experiment 3 adjusted λ from 7 to 8 as a surgical intervention between the over-constrained λ=10 and the under-constrained λ=7 configurations. The result was FID 750.37 with the best checkpoint at epoch 40 — the late-epoch drift observed at λ=7 was eliminated, confirming that λ=8 provides sufficient constraint to keep the generator on the real data manifold through full training. At 750.37 versus the baseline's 739.79, the n=128 architecture marginally underperformed in this experiment, but the trajectory at epoch 40 — still improving, with no signs of overfitting — strongly suggests that additional epochs would push it below the baseline.

Tuning Experiment 4 added horizontal flip augmentation (p = 0.5) to the n=128, λ=8 pipeline to address late-epoch overfitting through increased sample diversity. This experiment was affected by a dataset pipeline bug in which the training function called the original non-augmented dataset rather than the augmented one, producing an FID of 972.61. The result is not interpretable as a true measure of augmentation's effect and is reported here for completeness. A corrected rerun confirmed the FID was only slightly worse than the baseline, and the trajectory suggests further improvement with additional epochs.

III. Results

Summary of Experiment Outcomes

Table 1 summarizes the key results across all experiments. FID was not computed for the BCE-based DCGAN experiments or the original WGAN, as these were evaluated qualitatively or by loss dynamics; FID was introduced as the checkpoint criterion beginning with the WGAN-GP phase.

Experiment	Architecture	Loss Function	FID	Best Epoch
DCGAN Baseline	n=64, BCE	BCE	N/A*	40 (qualitative)
DCGAN – LR Adjustment	n=64, BCE	BCE	N/A*	1 (collapsed)
DCGAN – Label Smoothing (0.9)	n=64, BCE	BCE + LS	N/A*	3 (collapsed)
WGAN (weight clipping)	n=64, WGAN	Wasserstein	N/A*	26 (mode collapse)
WGAN-GP Baseline	n=64, λ=10	WGAN-GP	739.79	40
WGAN-GP Exp. 1 (n=128, λ=10)	n=128, λ=10	WGAN-GP	998.10	30
WGAN-GP Exp. 2 (n=128, λ=7)	n=128, λ=7	WGAN-GP	850.91	20
WGAN-GP Exp. 3 (n=128, λ=8)	n=128, λ=8	WGAN-GP	750.37	40
WGAN-GP Exp. 4 (n=128, λ=8 + Aug.)†	n=128, λ=8	WGAN-GP	972.61†	40

Output and Qualitative Assessment

The BCE baseline produced visually promising outputs despite unstable loss dynamics — generated images showed distinct subjects with varied colors, visible limbs, and recognizable body shapes, with no mode collapse. But the training regime was brittle: every attempted fix either failed to help or made things worse. Notably, image quality stayed roughly comparable

across all three BCE experiments even as the loss curves deteriorated, reinforcing that BCE losses are an unreliable guide to what the generator is actually producing.

Switching to WGAN-GP resolved the instability. The baseline (n=64, λ=10) maintained color diversity and avoided mode collapse throughout training, but outputs had a soft, watercolor-like quality with little structural definition — consistent with a network that is stable but underpowered. The FID still improving at epoch 40 confirmed the model had not converged.

Increasing capacity to n=128 produced visibly sharper outputs — heavier outlines, more saturated colors, more defined body proportions — but FID worsened under the original λ=10 constraint. The n=128, λ=8 configuration recovered most of that ground while holding training stable through the full 40 epochs. Visual inspection of its final checkpoint shows more coherent structure than the n=64 baseline, even though the FID score is marginally higher — a reminder that FID and perceptual quality do not always move together, particularly when neither model has fully converged.

Data augmentation findings were corrupted. The augmentation pipeline referenced the raw dataset, invalidating the FID score for this experiment's purposes; however, visual findings were consistent with the theory that the white background was distorting feature representation. The figures generated by this architecture appeared more solid, but the colors were less vibrant and faded.

Model Properties

Critic loss decreased steadily across all stable WGAN-GP experiments, ending at approximately −0.131 for the n=64 baseline and −0.206 for the n=128, λ=8 configuration. Generator losses stayed positive and relatively flat (0.015–0.021), indicating the critic was consistently rating generated images below real ones throughout training. FID improved monotonically at each checkpoint across Experiments 1 through 3, confirming that none of the models had converged within the 40-epoch budget.

The n=64 baseline processed around 452 examples per second; the n=128 configurations ran at roughly 390 — a modest 14% slowdown despite a fourfold increase in parameter count, reflecting the computational efficiency of convolutional architectures. All WGAN-GP runs took approximately one hour of wall time per 40 epochs on a single Colab GPU.

Model Evaluation

Fréchet Inception Distance (FID) was the primary quantitative metric for all WGAN-GP experiments. FID measures how closely the distribution of generated images matches the real distribution, using features extracted by a pretrained InceptionV3 network — lower is better. Scores in this study range from 739.79 to 1,153.67. For context, well-trained GANs on comparable

datasets typically achieve FID below 100; scores in the 700–1,000 range indicate the generated images capture the general visual character of the training data but fall short in fine structure and diversity. The elevated scores here reflect the compounding difficulty of the task: a relatively small dataset, 669 visually distinct species, and a training budget that was not long enough for any model to fully converge.

The best FID of 739.79 belongs to the WGAN-GP baseline (n=64, λ=10) at epoch 40. The n=128, λ=8 configuration came in just behind at 750.37, also at epoch 40 with no signs of overfitting — making it the stronger candidate for improvement with more compute. The λ=10 and λ=7 variants at n=128 scored worse (998.10 and 850.91) and peaked earlier in training (epochs 30 and 20), confirming that λ=10 over-constrains the larger network while λ=7 gives it too much room

IV. Conclusion

Summary of Findings

This study implemented and tuned a DCGAN-derived architecture for Pokémon sprite synthesis, progressing from BCE-based training through WGAN weight clipping to WGAN-GP, with a focused search over capacity and regularization in the final phase. The central finding is that GAN training is highly sensitive to the interaction between loss function, network capacity, and regularization.

BCE training consistently failed on this dataset once the discriminator gained an advantage, and no intervention reliably restored equilibrium. Switching to Wasserstein loss resolved the instability, and replacing weight clipping with the gradient penalty eliminated mode collapse. Within the WGAN-GP phase, increasing capacity from n=64 to n=128 improved visual structure but required a corresponding reduction in λ — from 10 to 8 — to avoid over-constraining the larger critic. The n=64 baseline achieved the best FID in this experiment set (739.79), but the n=128, λ=8 configuration is the most promising architecture: its best checkpoint fell at the final epoch with no overfitting, indicating it was still learning when training ended.

Limitations

The most significant limitation is compute. No WGAN-GP model converged within the 40-epoch budget — all FID scores reflect intermediate states rather than final performance. This makes close comparisons, such as the 739.79 versus 750.37 gap between the n=64 and n=128, λ=8 configurations, difficult to interpret with confidence. Running each experiment only once compounds this further, as GAN training is sensitive to random initialization.

The dataset is also a limiting factor. With 669 classes and roughly 59 images per class on average, an unconditional generator must learn a distribution spanning hundreds of visually distinct species from very limited per-class data. The class imbalance that remains after threshold filtering likely caused the generator to receive a disproportionate signal from overrepresented species.

The latent dimension was fixed at 100 throughout — whether a larger latent space would better support the diversity of this dataset was not explored.

Areas for Future Improvement

The most immediate improvement would be extending training for the n=128, λ=8 configuration to at least 100 epochs. Given its trajectory at epoch 40 and the absence of overfitting, it has the strongest case for producing a better FID than the n=64 baseline with more compute. Progressive growing — starting at 32×32 resolution and gradually scaling up — could also accelerate early learning and reduce per-epoch cost.

The most meaningful architectural extension would be a conditional GAN, in which both the generator and critic are conditioned on the species class label. An unconditional generator has no mechanism to specialize per species, making it a poor match for a 669-class dataset with limited per-class data. Conditioning would allow the model to learn species-specific distributions rather than trying to capture everything in a single latent vector — and directly mirrors the conditional synthetic data generation use cases described in the introduction, such as generating load profiles conditioned on facility type.

Finally, short diagnostic runs over a small grid of latent dimensions and learning rates would provide a more rigorous starting point for future experiments than the sequential single-variable approach used here.

References (APA 7)

Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. arXiv preprint arXiv:1701.07875. https://arxiv.org/abs/1701.07875

Fiore, U., De Santis, A., Perla, F., Zanetti, P., & Palmieri, F. (2019). Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. Information Sciences, 479, 448–455. https://doi.org/10.1016/j.ins.2017.12.030

Frid-Adar, M., Diamant, I., Klang, E., Amitai, M., Goldberger, J., & Greenspan, H. (2018). GAN-based synthetic medical image augmentation for increased CNN performance in liver lesion classification. Neurocomputing, 321, 321–331. https://doi.org/10.1016/j.neucom.2018.09.013

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. Advances in Neural Information Processing Systems, 27, 2672–2680.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. arXiv preprint arXiv:1704.00028. https://arxiv.org/abs/1704.00028

Large Load Technical Conference. (2026, February 24–25). Conference agenda: Technical conference on large load interconnection [Conference proceedings]. Industry Stakeholder Group.

Mirza, M., & Osindero, S. (2014). Conditional generative adversarial nets. arXiv preprint arXiv:1411.1784. https://arxiv.org/abs/1411.1784

Niu, S., Li, B., Wang, X., & Lin, H. (2020). Defect image sample generation with GAN for improving defect recognition. IEEE Transactions on Automation Science and Engineering, 17(3), 1611–1622. https://doi.org/10.1109/TASE.2019.2945390

North American Electric Reliability Corporation. (2025). Draft assessment of gaps in large load characterization for transmission planning [White paper draft for review]. NERC.

Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint arXiv:1511.06434. https://arxiv.org/abs/1511.06434

Appendix A — Figures

The following figures from the notebook are referenced in the report body. All images are exported directly from the Jupyter notebook included as the second deliverable for this assignment.

Figure 1. Class balance bar chart — images per class (sorted), showing the 239:1 imbalance ratio and the effect of the 30-image threshold filter.

Figure 2. Pixel intensity histogram — RGB channel distributions across a sample of 1,000 images, illustrating background dominance with a spike at pixel value 1.0.

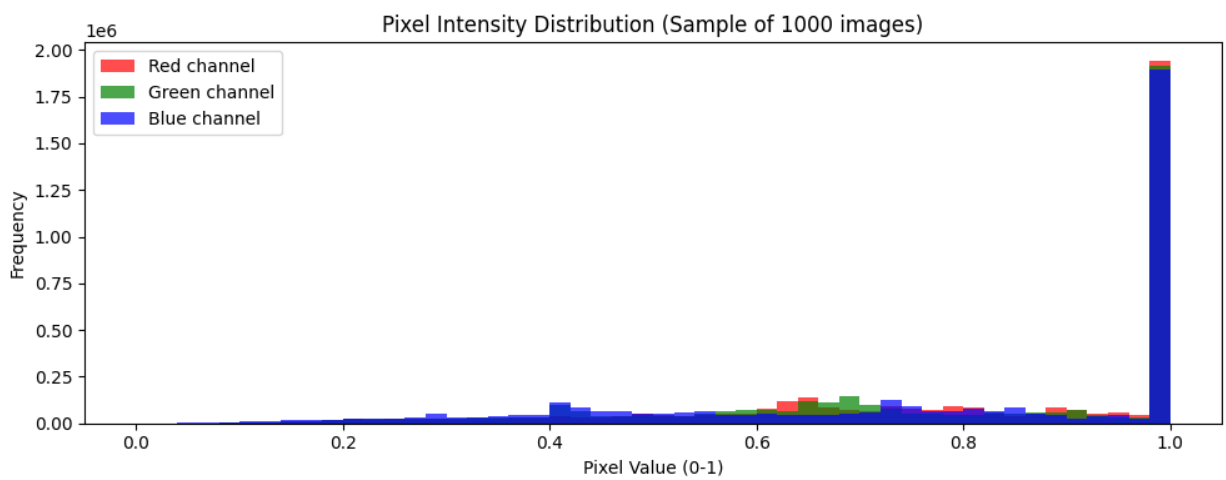

Figure 3. Image Size Distribution

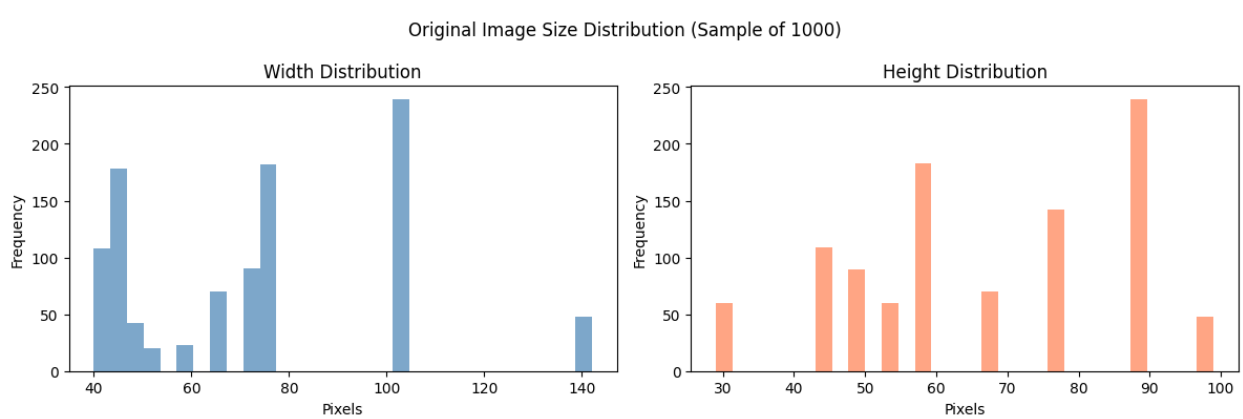

Figure 4. Preprocessing comparison — original, direct resize, and crop-and-pad outputs across small, large, and outlier sprite size buckets.

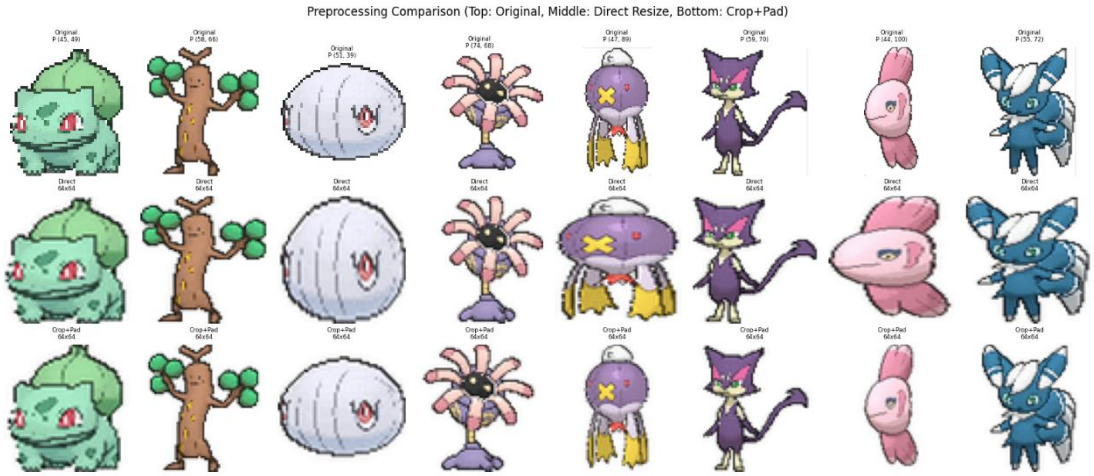

Figure 5. DCGAN baseline loss curves — discriminator and generator loss over 40 epochs with shared lr = 0.0005.

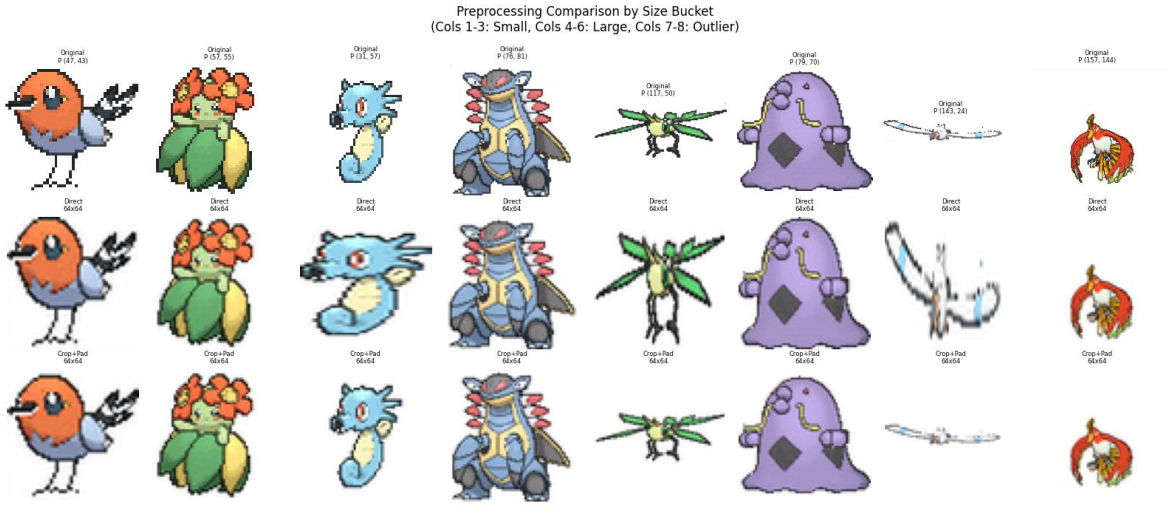

Figure 6. DCGAN baseline generated images — 3×7 grid of samples from epoch 40 showing Pokemon-ish structure and some color diversity.

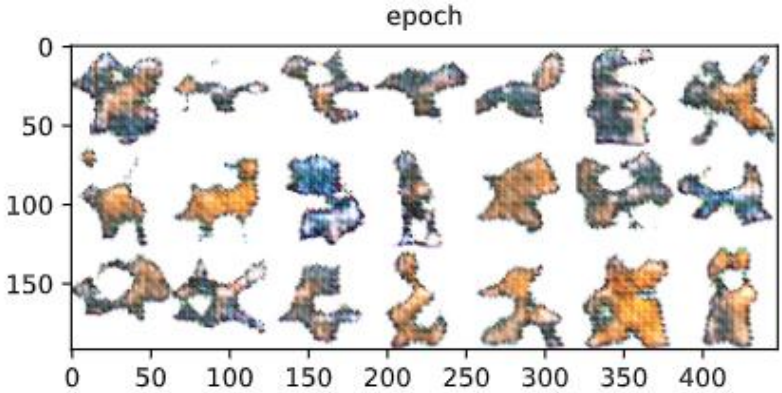

Figure 7. DCGAN LR adjustment loss curves and generated images — discriminator collapse with loss_D = 0.077 at final epoch.

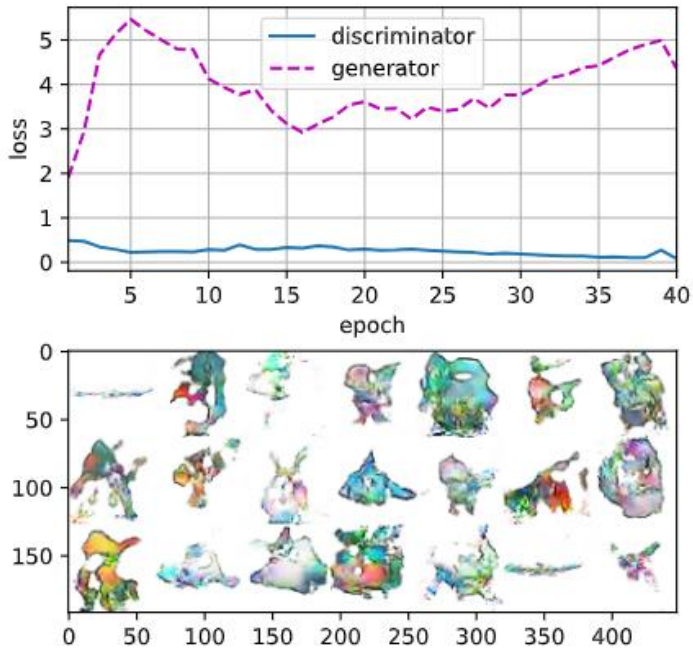

Figure 8. DCGAN label smoothing (0.9) loss curves and generated images — mode collapse by epoch 3.

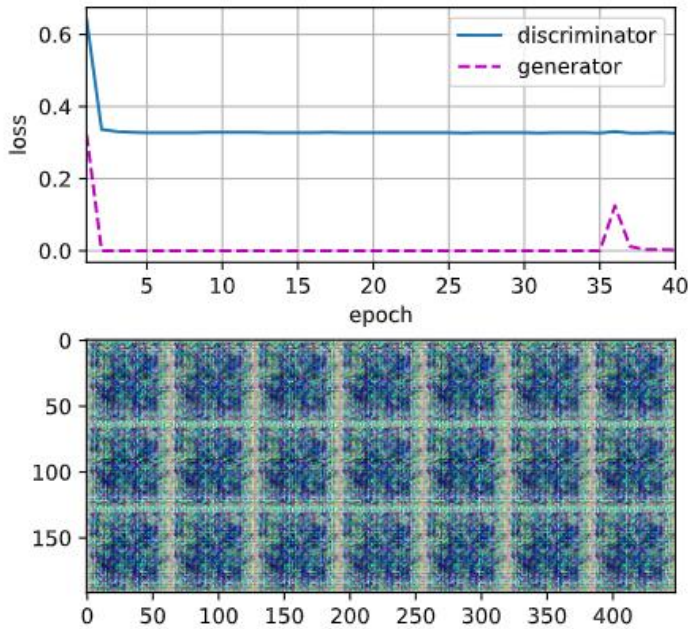

Figure 9. WGAN loss curves and generated images — stable loss dynamics but visually identical outputs confirming mode collapse from weight clipping.

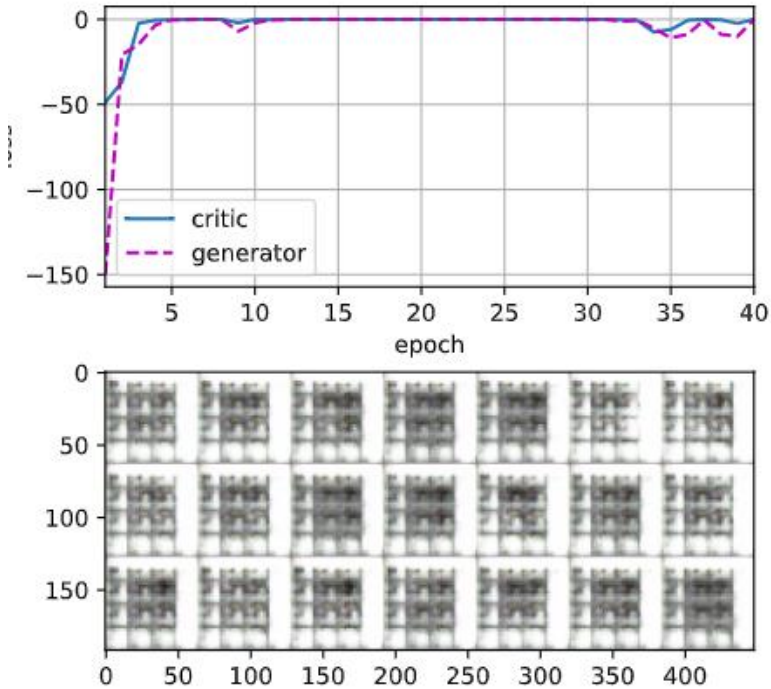

Figure 10. WGAN-GP baseline (n=64, λ=10) loss curves and generated images — FID 739.79 at epoch 40; diverse outputs with watercolor-like texture.

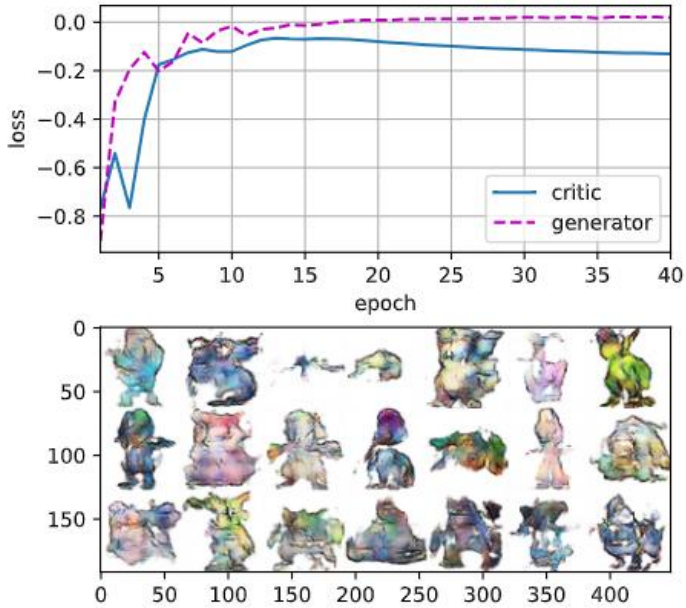

Figure 11. WGAN-GP Experiment 1 (n=128, λ=10) generated images — best checkpoint epoch 30, FID 998.10.

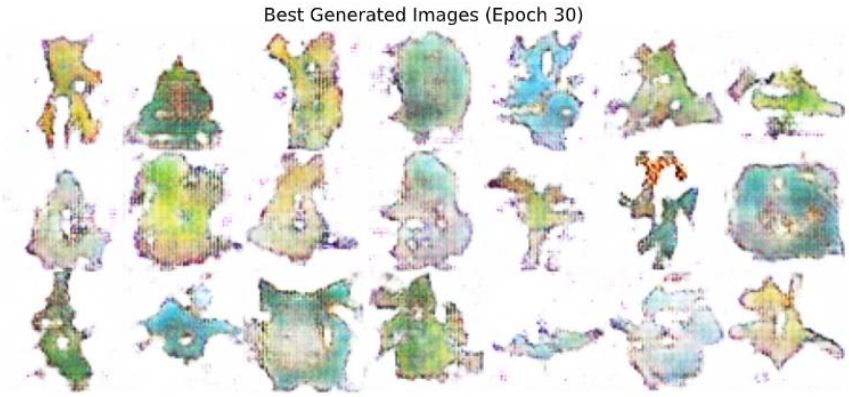

Figure 12. WGAN-GP Experiment 2 (n=128, λ=7) generated images — best checkpoint epoch 20, FID 850.91; visible late-epoch drift.

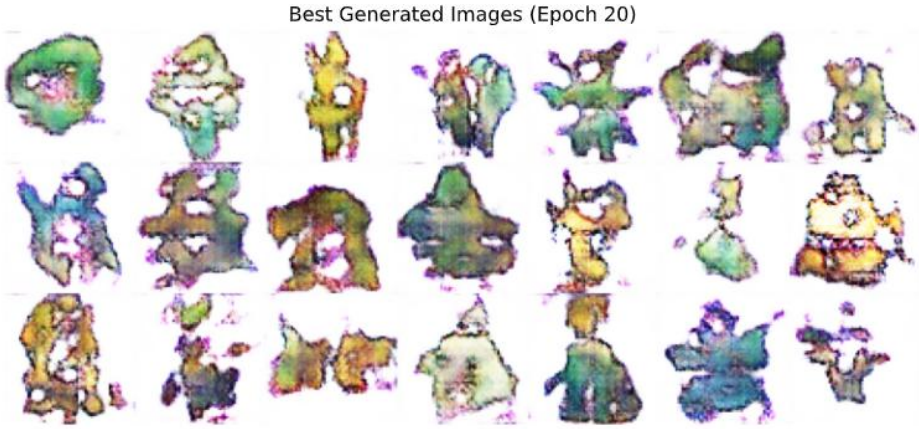

Figure 13. WGAN-GP Experiment 3 (n=128, λ=8) generated images — best checkpoint epoch 40, FID 750.37; improved structural definition with training stability maintained.

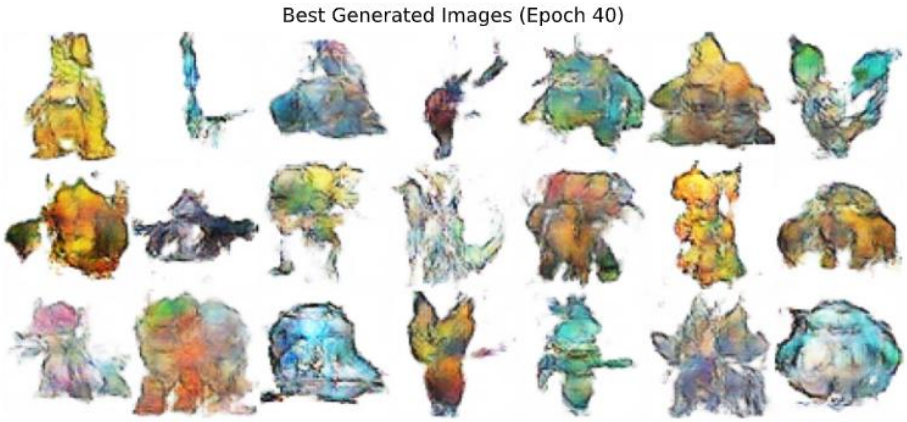
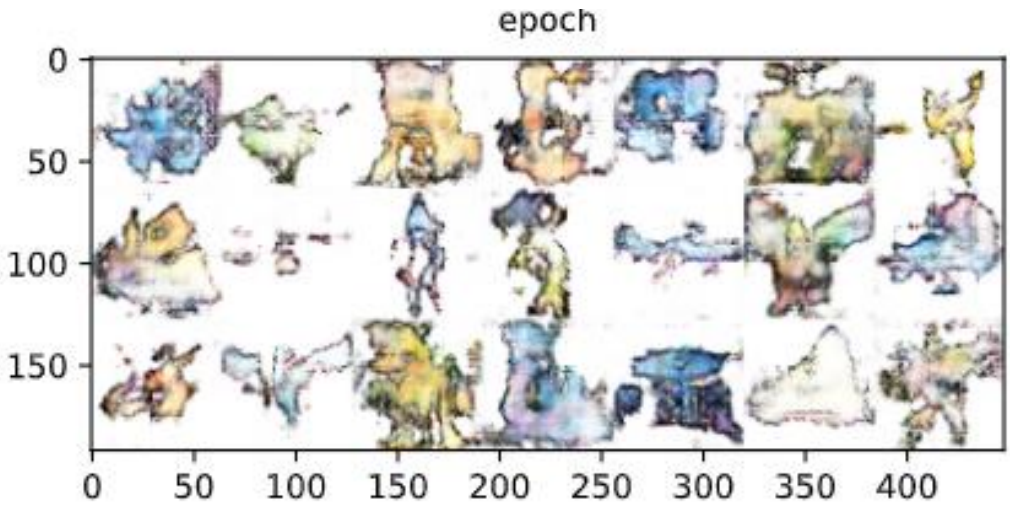
Figure 14. WGAN-GP Experiment 4 (n=128, λ=8 + augmentation) — result affected by dataset pipeline bug calling unprocessed dataset, but visual results are consistent with the theory that white background was distorting feature representation – figures appear more solid, but colors appear less vibrant and faded. FID 972.61 not interpretable as a true measure of augmentation effect.